Nouveaux comportements dynamiques

Voici une description des nouveaux comportements dynamiques côté client qui vous sont demandés. Vous êtes libres d'en modifier le style et la mise en page, mais vous devez absolument en respecter l'ergonomie.

Important

Les dynamismes décrits ci-dessous doivent être implémentés du côté client en JavaScript, jQuery et AJAX en fonction des besoins.

Le cas échéant, vous développerez également les services nécessaires du côté serveur ; ceux-ci échangeront des données avec la partie cliente exclusivement en JSON (et donc pas en HTML). Respectez la FAQ concernant les règles de conception des services.

Pour cette itération, vous ne pouvez pas utiliser d'autre librairie JavaScript que jQuery, jQuery UI, jQuery UI Touch Punch (pour le *drag & drop*), les fenêtres modales et le carousel de Bootstrap.

Toutes les fonctionnalités de la première itération doivent continuer de fonctionner comme avant si JavaScript est désactivé (voir plus loin).

Rappel

Constantes de validation : les limites de validation (longueurs min/max, etc.) utilisées côté client ne doivent **pas** être codées en dur dans le JavaScript. Elles doivent être récupérées via un **service AJAX** dédié (ex. un service qui renvoie les valeurs de configuration au format JSON). Le client appelle ce service au chargement (ou avant les validations) et utilise les valeurs reçues pour les contrôles synchrones et asynchrones. Côté serveur (PHP), ces mêmes constantes sont lues depuis la configuration (ex. `Configuration::get`).

A. Recherche *fulltext* dans la liste des annonces

Si JavaScript est activé, un champ de recherche textuel est affiché au-dessus de la page *browse items* et de la page *my items*. L'utilisateur saisit un motif ; la liste affichée est filtrée pour ne montrer que les annonces (accessibles à l'utilisateur courant) dont au moins une information (titre, description, auteur, etc.) contient ce motif.

Les annonces restent triées par ordre croissant de date de fin. S'il n'y a aucun résultat, le message "No item found." est affiché. L'URL de la page ne change pas pendant la saisie dans le

champ de recherche ; le filtrage est effectué côté client via un service, sans passer le motif en paramètre GET dans l'URL.

B. Propagation du critère de recherche

Lorsqu'on ouvre une annonce (*Open*) depuis la liste (*browse* ou *my items*), puis qu'on navigue depuis cette page (enchère, édition, gestion des images, etc.), le bouton *back* doit permettre de revenir à la liste dans le même état qu'au moment du départ : même motif de recherche affiché et appliqué si une recherche était active. Depuis une page d'édition ou de gestion des images, le *back* ramène d'abord sur la page de l'annonce ouverte, puis depuis celle-ci un *back* ramène sur la liste ; l'état de recherche est ainsi conservé tout au long de la navigation.

Ce comportement doit **obligatoirement** être implémenté sans utiliser `$_SESSION` (interdite car partagée entre les onglets) ni les cookies. Pour cela, l'état de la recherche (par ex. le motif) est encodé et passé en paramètre dans l'URL vers les pages accessibles depuis l'annonce (item/open, item/edit, etc.). Lors d'un *back*, l'URL de la liste (browse ou my items) contient ce paramètre ; au chargement, la liste restaure le motif et réapplique le filtre.

Le framework limite à 3 paramètres par URL et n'accepte pas certains caractères (ex. accentués). Pour encoder l'état dans un seul paramètre compatible URL, vous utiliserez les méthodes utilitaires suivantes (à ajouter à une classe utilitaire) pour *encoder / décoder* un objet (ex. tableau associatif avec le motif) en chaîne base64url :

```
/**
 * Permet d'encoder un string au format base64url, c'est-à-dire un format
base64 dans lequel
 * les caractères '+' et '/' sont remplacés respectivement par '-' et '_', ce
qui permet d'utiliser le
 * résultat dans une URL.
 * @param string $data Le string à encoder.
 * @return string Le string encodé.
 */
private static function base64url_encode(string $data) : string {
    return rtrim(strtr(base64_encode($data), '+/', '-_'), '=');
}

/**
 * Permet de décoder un string encodé au format base64url.
 * @param string $data Le string à décoder.
 * @return string Le string décodé.
 */
private static function base64url_decode(string $data) : string {
    return base64_decode(strtr($data, '-_', '+/') . str_repeat('=', 3 - (3 +
strlen($data)) % 4));
}

/**
 * Permet d'encoder une structure de donnée (par exemple un tableau associatif
ou un objet) au format base64url.
 * @param mixed $data La structure de données à encoder.
```

```
     * @return string Le string résultant de l'encodage.
     */
    public static function url_safe_encode(mixed $data) : string {
        return self::base64url_encode(gzcompress(json_encode($data), 9));
    }

    /**
     * Permet de décoder un string au format base64url.
     * @param string Le string à décoder.
     * @return mixed $data La structure de données décodée.
     */
    public static function url_safe_decode(string $data) : mixed {
        return json_decode(@gzuncompress(self::base64url_decode($data)), true,
    512, JSON_OBJECT_AS_ARRAY);
    }
```

Pour illustrer ce principe, voici un exemple d'un scénario de navigation, avec la succession des URL associés à chaque étape[1]:

1. J'ouvre la liste des annonces :

   `GET http://localhost/prwb_2526_b00/item/browse`

2. Je saisis "brown" dans le champ de recherche et je clique sur l'annonce "Clavier mécanique Keychron K2" (`21`) :

   `GET http://localhost/prwb_2526_b00/item/open/21/from-browse/eNpTqswvLVICAAdQAhQ`

   Le dernier paramètre est l'état de recherche encodé (motif "brown").

3. Je fais une ou plusieurs actions (ex. enchères, consultation d'images, etc.) ; après un éventuel PRG, je me retrouve en GET sur la page de l'annonce ouverte, l'URL contenant toujours le paramètre encodé :

   `GET http://localhost/prwb_2526_b00/item/open/21/from-browse/eNpTqswvLVICAAdQAhQ`

4. Je clique sur l'icône "back" et je reviens sur la liste filtrée avec le même motif appliqué :

   `GET http://localhost/prwb_2526_b00/item/browse/eNpTqswvLVICAAdQAhQ`

C. Popup de confirmation à la sortie du formulaire d'ajout ou d'édition d'une annonce

Lorsque l'utilisateur tente de quitter la page du formulaire d'ajout ou d'édition d'une annonce (par un lien, le bouton *back*, ou toute navigation) alors que le formulaire n'est pas *pristine* (il a été modifié depuis le chargement), une modale Bootstrap doit demander une confirmation (par ex. "Leave with unsaved changes?") — voir Modal - Bootstrap 5.3 (Via JavaScript). Si l'utilisateur confirme, la navigation a lieu ; sinon, il reste sur le formulaire. Un formulaire pristine (aucune modification) ne déclenche pas la modale.

D. Validations synchrones et asynchrones lors de l'ajout et de l'édition d'une annonce

Les validations (synchrones et asynchrones) des pages d'ajout et d'édition d'une annonce sont gérées dynamiquement côté client. Les appels AJAX ne sont utilisés que lorsque nécessaire (par ex. pour vérifier l'unicité du titre pour l'utilisateur courant) ; les autres règles (obligatoire, longueur, format) sont vérifiées de façon synchrone. **Les constantes de validation (longueurs min/max, etc.) ne doivent pas être en dur dans le JS : elles sont récupérées via le service AJAX de configuration** (voir rappel en tête de document). Les mêmes règles doivent être appliquées côté serveur (PHP) pour le cas où JavaScript est désactivé.

Les validations sont déclenchées à la saisie (ou à la sortie du champ). Les messages d'erreur s'affichent sous le champ concerné ; le cadre du champ (groupe input, ex. label + input) est marqué en rouge en cas d'erreur et en vert si le champ est valide. Le bouton de sauvegarde est désactivé tant qu'il y a au moins une erreur. **L'indication visuelle (rouge/vert) et les messages ne s'affichent pour un champ que lorsque l'utilisateur a interagi avec ce champ (saisie, `blur`) ou a tenté de soumettre le formulaire** ; les champs non touchés restent neutres.

Tout se fait sur une seule page, sans rechargement.

E. *Drag & drop* des images

Dans la vue "Manage Images", on doit pouvoir modifier l'ordre des images en les tirant avec la souris et en les lâchant à l'endroit souhaité. Pendant le *drag*, le déplacement de l'image et le réarrangement dynamique des autres vignettes doivent être visibles.

Lorsqu'on lâche l'image, elle reste à l'endroit choisi et l'ordre des images est mis à jour côté client **sans rechargement de la page** ; en parallèle, un service côté serveur met à jour les priorités en base de données pour refléter ce nouvel ordonnancement.

Le *drag & drop* doit être implémenté avec la librairie jQueryUI (voir https://jqueryui.com/sortable/).

> **Remarque**
>
> La librairie jQuery UI ne prend pas en charge les événements "tactiles" (*touch events*), ce qui explique que le *drag & drop* ne fonctionne pas correctement lorsqu'on est en mode "mobile". Pour le faire fonctionner, il suffit d'inclure la librairie "jQuery UI Touch Punch" (voir https://github.com/RWAP/jquery-ui-touch-punch).

Sans JavaScript, la page "Manage Images" conserve le comportement de base (aucun *drag & drop*), et les actions existantes continuent de fonctionner via le flux POST-Redirect-GET.

F. Carousel et clic sur vignette (page *open item*)

Sur la page d'affichage d'une annonce (*open item*), si JavaScript est activé, les images de l'annonce sont affichées dans un carousel (voir Carousel - Bootstrap 5.3). Un clic sur une vignette affiche l'image correspondante en grand dans la zone principale **sans rechargement de la page**.

Les URLs des images ne doivent pas être construites ni manipulées côté client : un **service AJAX** fournit la liste des images (chemins ou URLs utilisables en `src` des balises `<img>`). Le client utilise uniquement ces données pour construire le carousel et les vignettes.

Sans JavaScript, le comportement reste inchangé : une image principale et, s'il y en a plusieurs, une section de vignettes ; chaque vignette est un lien dont le clic recharge la page avec l'image choisie.

G. Gestion dynamique des boutons *left* / *right* et suppression d'une image

Dans la vue *Manage Images*, lorsque JavaScript est activé, les boutons *left* et *right* doivent réordonner les images immédiatement dans la page, sans rechargement. À chaque clic, l'ordre affiché est mis à jour côté client et un service côté serveur met à jour les priorités en base de données pour refléter ce nouvel ordonnancement.

Le bouton de suppression d'une image doit, avec JavaScript, ouvrir une modale Bootstrap de confirmation (voir Modal - Bootstrap 5.3 (Via JavaScript)). Si l'utilisateur confirme, l'image est supprimée via un service côté serveur (fichiers et enregistrement associés) et disparaît immédiatement de la page, sans *refresh* ; s'il annule, rien ne change.

Sans JavaScript, les boutons conservent leur comportement initial basé sur des formulaires POST et le flux POST-Redirect-GET.

H. Validations lors de l'inscription et de l'édition du profil

Les validations décrites ci-dessous concernent **la page d'inscription (signup)** et **la page d'édition du profil (edit profile)**. Les mêmes principes d'ergonomie s'appliquent aux deux pages.

Signup : validations synchrones et asynchrones (disponibilité de l'email, du pseudo et du nom complet via AJAX ; champs obligatoires, longueurs, format email, règles du mot de passe, confirmation non vide et identique au mot de passe en synchrone). En PHP (PRG), les erreurs sont affichées sous chaque champ concerné (pas de bloc d'erreurs global).

Edit profile : validations synchrones et asynchrones sur les champs full name, pseudo, email et IBAN (disponibilité email / pseudo / full name via AJAX en excluant l'utilisateur courant ; obligatoire, longueurs, format email, format IBAN optionnel en synchrone). En PHP (PRG), les erreurs sont affichées sous chaque champ concerné (pas de bloc d'erreurs global).

Constantes de validation : pour *signup* et *edit profile*, les limites (longueurs min/max, etc.) ne doivent pas être codées en dur dans le JavaScript ; elles sont récupérées via le service AJAX de configuration (voir rappel en tête de document).

Pour les deux pages : les validations sont déclenchées à la saisie (ou à la sortie du champ). Les messages d'erreur s'affichent sous le champ concerné ; le cadre du champ est marqué en rouge en cas d'erreur et en vert si le champ est valide. Le bouton de soumission est désactivé tant qu'il y a au moins une erreur. **L'indication visuelle (rouge/vert) et les messages ne s'affichent pour un champ que lorsque l'utilisateur a interagi avec ce champ (saisie, `blur`) ou a tenté de soumettre le formulaire** ; les champs non touchés restent neutres. Tout se fait sur une seule page, sans rechargement.

---

1. Les URL seront bien évidemment différents dans votre application, mais ce scénario illustre le principe du fonctionnement demandé. ↩